

Atividade



Teste enviado



- Entrega Sem prazo
- Pontos 15
- Perguntas 5
- Limite de tempo Nenhum
- Tentativas permitidas Sem limite

Instruções

O objetivo desta atividade é proporcionar uma melhor fixação quanto ao conteúdo da unidade 4. Com isso você será capaz de ter um entendimento das abordagens apresentadas na unidade.

Para iniciar o questionário:

- Clique em "**Fazer o teste**";
- Responda todas as questões e depois clique no botão "**Enviar teste**" para finalizar a atividade.

[Fazer o teste novamente](#)

Histórico de tentativas

	Tentativa	Tempo	Pontuação
MAIS RECENTE	Tentativa 1	3 minutos	15 de 15

❗ As respostas corretas estão ocultas.

Pontuação desta tentativa: 15 de 15

Enviado 7 jan em 19:06

Esta tentativa levou 3 minutos.



Pergunta 1

3 / 3 pts

O módulo de cache do NestJS permite o uso de interceptadores e TTL (time-to-live) para controlar o tempo de armazenamento em memória dos dados armazenados.

As estratégias de cache no NestJs podem ser implementadas utilizando decorators e funções do cache management. Em relação ao uso de decorators, qual a função do `@CacheTTL(30000)` nos endpoints?

- ☐ O tempo máximo da requisição será de 30 segundos.
- ☐ O cache será mantido indefinidamente.
- ☒ Os dados armazenados no cache daquele endpoint expiram após 30 segundos.
- ☐ A requisição será executada em exatamente 30 segundos
- ☐ Nenhuma das alternativas.



Pergunta 2

3 / 3 pts



Teste enviado

Sobre o sistema de cache implementado na aplicação NestJs, assinale a alternativa **CORRETA**:

☐ O único sistema de cache possível de ser implementado em aplicações Nest é o cache em memória



O sistema de cache pode ter sua implementação abstraída, de forma com que possa ser possível trocar o banco de cache sem tantas alterações na regra de negócio

☐ O único sistema de cache possível de ser implementado em aplicações Nest é o cache utilizando o Redis



O sistema de cache é estático, portanto a abstração em cima da implementação não é suficiente para fazer com que alterações no banco do cache aconteçam sem muitas alterações na regra de negócio

☐ Nenhuma das alternativas



Pergunta 3

3 / 3 pts

A comunicação entre microsserviços pode ser feita via TCP ou por meio de mensagens em sistemas como Kafka ou RabbitMQ. A escolha afeta diretamente a arquitetura, resiliência e escalabilidade da aplicação.

Com base nos diferentes estilos de comunicação entre microserviços (request/response e event-based), qual das opções descreve corretamente o padrão event-based?

☐ Deve ser evitado em aplicações que utilizam múltiplos bancos de dados.



Garante que processos sejam sempre executados em uma mesma ordem, não importando quantos microsserviços estejam ouvindo os eventos que serão publicados

☐ É utilizado quando queremos garantir resposta obrigatória para cada evento emitido.



Permite publicar eventos para diversos serviços ouvirem sem a necessidade de resposta direta, favorecendo o desacoplamento e a escalabilidade.

☐ Utiliza HTTP síncrono para garantir que todos os serviços recebam a mesma mensagem.



Pergunta 4

3 / 3 pts

O NestJS oferece suporte nativo a sistemas de cache que podem ser utilizados para otimizar o desempenho de aplicações, especialmente em endpoints com consultas custosas. Além do cache in-memory, também é possível configurar sistemas como Redis para armazenar os dados em memória distribuída.

Qual é o impacto de utilizar cache Redis em aplicações NestJS e em quais cenários a sua utilização é mais indicada?

☐ Redis substitui



Teste enviado

☐ O Redis é indicado quando temos múltiplas instâncias (ou microsserviços) da aplicação rodando, pois facilita o compartilhamento de cache

☐ O uso de Redis é obrigatório sempre que usamos NestJS com TypeORM.

☐ O uso de cache torna desnecessário o uso de bancos de dados, pois armazena todos os dados necessários.

☐ Redis é útil apenas para aplicações que não utilizam bancos de dados relacionais.



Pergunta 5

3 / 3 pts

Em arquiteturas distribuídas, a escolha entre protocolos de comunicação pode afetar diretamente a escalabilidade e a resiliência do sistema. O uso de TCP é direto e simples, enquanto mensageria com ferramentas como Kafka e RabbitMQ introduz novos padrões como publish/subscribe e event-driven.

Com base nos padrões de comunicação entre microserviços, quais são as principais implicações práticas da adoção de sistemas de mensageria em comparação com comunicação via TCP?



TCP é preferido para sistemas altamente escaláveis, por não depender de intermediários e possibilitar a utilização de controles de throughput.

☐ TCP oferece melhor escalabilidade e desacoplamento que mensageria.



Ambos são equivalentes em arquiteturas modernas, e a escolha depende apenas da linguagem de programação usada.



Mensageria é mais robusta e escalável, permitindo comunicação assíncrona entre serviços, embora adicione complexidade e demanda uma infraestrutura extra.

☐ Mensageria é mais rápida e exige menos infraestrutura.

Pontuação do teste: 15 de 15